



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Smart Electric Vehicle Charging Network Management Platform

Test Case Design Document

EVCP Booking & Station Service

<https://github.com/RAKS-rak57/evcp-booking-service->

Submitted in the partial fulfilment for the Course of

CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of

**BACHELOR OF ENGINEERING IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

Submitted by

R. R. RAKSHITH (192324156)

Under the Supervision of

Dr. K. Anitha Davamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

JUL 2026



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

I, R. R. Rakshith, of the Department of Artificial Intelligence & Data Science, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Case study work entitled **Smart Electric Vehicle Charging Network Management Platform** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 14/08/2026

Signature of the Students with Names

R R RAKSHITH (192324156)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, **Dr. Anusuya** for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr. K. Anitha Davamani** for her creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

R R RAKSHITH (192324156)

TABLE OF CONTENTS

S.NO	TOPIC	PAGE NO.
1	REQUIREMENTS UNDER TEST	3
2	TEST SCENARIOS IDENTIFIED	4
3	TEST DESIGN TECHNIQUES APPLIED	5-6
4	DETAILED TEST CASES	7-11
5	REQUIREMENT TRACEABILITY MATRIX	12
6	TEST EXECUTION EVIDENCE	13-14
7	TEST COVERAGE SUMMARY AND DEFECT FINDINGS	15-16

1. Requirements Under Test

This document designs and executes test cases against the EVCP Booking & Station Service — the same Node.js/Express microservice built in CO3-AT1 and reviewed in CO3-AT2. All test cases in this document were actually executed against a live instance of the service (real PostgreSQL, real Redis, real Express server), not written as hypothetical expectations only; the Actual Result and Status columns reflect genuine execution output, independently reproduced twice — once in the report-preparation session and once by the student directly on their own machine using Docker Desktop (Section 6).

1.1 Functional Requirements Under Test

ID	Functional Requirement	Endpoint
FR-02	The system shall allow an EV owner to search for charging stations within a specified radius of their location.	GET /api/stations
FR-03	The system shall display real-time availability of charging connectors at each station.	GET /api/stations/:id/availability
FR-04	The system shall allow an EV owner to reserve a charging slot in advance, with a defined hold period.	POST /api/bookings
FR-04a	The system shall allow a booking's status to be retrieved.	GET /api/bookings/:id

1.2 Non-Functional Requirements Under Test

Quality Attribute	Requirement Under Test
Reliability	Invalid or malformed input must be rejected with a clear, safe error response rather than crashing the service or leaking internal details.
Performance	Repeated availability lookups for the same station within a short window should be served from cache rather than re-querying the database.

Data Consistency	A reserved slot must not remain visible as available to other users, and a connector must not be double-booked while already reserved.
Security	Error messages returned to the client must not expose raw database internals (SQL error text, constraint names).
Deployment Reliability	The containerized environment (docker-compose) must reliably (re)initialize its schema on a fresh deployment.

2. Test Scenarios Identified

Scenario ID	Scenario Description	Related Requirement
TS-01	Search for stations with valid and invalid location parameters, across normal, boundary, and invalid radius values.	FR-02
TS-02	Retrieve real-time availability for valid, invalid, and non-existent stations, and verify caching behavior.	FR-03
TS-03	Create a booking with complete and incomplete payloads (decision table across 3 required fields).	FR-04
TS-04	Attempt to book a connector that does not exist, and a connector that is already reserved.	FR-04, Data Consistency
TS-05	Retrieve booking status for valid, non-existent, and malformed booking IDs.	FR-04a
TS-06	Verify the booking hold expires and transitions to a released state after its hold window passes.	FR-04 (state transition)
TS-07	Verify that the availability cache is correctly invalidated immediately after a new booking is made.	FR-03, FR-04 integration
TS-08	Verify the containerized deployment correctly initializes its database schema on a fresh docker-compose run.	Deployment Reliability

3. Test Design Techniques Applied

3.1 Equivalence Partitioning (EP)

Input parameters were partitioned into valid and invalid classes, with at least one representative test case drawn from each partition.

Input	Valid Partition	Invalid Partition(s)
lat / lng	Numeric decimal within real-world coordinate range (e.g., 13.08, 80.21)	Missing; non-numeric string ("abc")
stationId (path param)	Existing integer ID (1, 2)	Non-existent integer ID (999); non-numeric ("abc")
Booking payload fields	All of userId, stationId, connectorId present	One or more fields missing

3.2 Boundary Value Analysis (BVA)

Boundary values were tested around the radius search parameter and the booking hold-window duration.

Boundary	Values Tested
Search radius	0 (exact-location-only boundary), a very large value (10000 km, effectively unbounded), and a negative value (-5, invalid boundary)
Booking hold window	Exactly 15 minutes — verified hold_expires_at equals start_time + 15 minutes precisely (TC-16)
Station/booking ID	0 and a large out-of-range ID (999, 9999) as lower/upper edge cases around valid existing IDs (1-2)

3.3 Decision Table Testing

The booking-creation payload (userId, stationId, connectorId) was tested using a decision table covering the presence/absence of each required field.

Rule	userId present?	stationId present?	connectorId present?	Expected Outcome

R1	Y	Y	Y	201 Created — booking reserved
R2	N	Y	Y	400 Bad Request — validation error
R3	Y	N	Y	400 Bad Request — validation error
R4	Y	Y	N	400 Bad Request — validation error
R5	N	N	N	400 Bad Request — validation error

3.4 State Transition Testing

The Booking entity's lifecycle was modelled and tested as a state machine: No Booking → RESERVED (on successful POST) → RELEASED (once the hold window expires). The diagram below also documents an invalid transition the current implementation incorrectly permits.

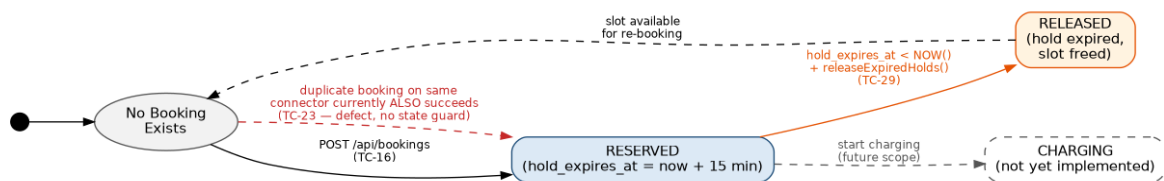


Figure 3.1: Booking State Transition Diagram

4. Detailed Test Cases

All test cases below were executed against a live instance of the service (real PostgreSQL + Redis + Express, running locally). PASS indicates actual output matched the expected/specified behavior; FAIL indicates a genuine defect was found — these are discussed further in Section 7. Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Status are presented as seven distinct fields for every case, as required.

4.1 Station Search — GET /api/stations (FR-02)

ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-01	Normal: search near an existing station	1) Send GET /api/stations with valid lat, lng, radius	lat=13.08, lng=80.21, radius=10	200 OK, 1 station returned	200 OK, count:1, "Anna Nagar Charging Hub" returned	PASS
TC-02	Normal: wide radius covering 2 stations	1) Send GET /api/stations with a wider radius	lat=13.03, lng=80.23, radius=50	200 OK, 2 stations returned	200 OK, count:2, both stations returned	PASS
TC-03	Boundary: radius=0 at exact coordinates	1) Send GET with radius=0 at a station's exact lat/lng	lat=13.085, lng=80.2101, radius=0	200 OK, exact-match station returned	200 OK, count:1 (distance=0 satisfies <=0)	PASS
TC-04	Boundary: very large radius	1) Send GET with an unrealistically large radius	radius=10000	200 OK, all stations returned	200 OK, count:2, both stations returned	PASS
TC-05	Invalid: missing lat	1) Send GET omitting the lat parameter	lng=80.21, radius=10 (no lat)	400 Bad Request	400, "lat and lng query parameters are required"	PASS
TC-06	Invalid: missing lng	1) Send GET omitting the lng parameter	lat=13.08, radius=10 (no lng)	400 Bad Request	400, same validation message	PASS
TC-07	Invalid: missing both lat and lng	1) Send GET omitting both parameters	radius=10 only	400 Bad Request	400, same validation message	PASS
TC-08	Exceptional: non-numeric lat	1) Send GET with lat set to a text string	lat=abc, lng=80.21, radius=10	400 Bad Request with a safe validation message	500 Internal Server Error, raw Postgres error text ("invalid input syntax for type double	FAIL

					precision") leaked to client	
TC-09	Invalid: negative radius	1) Send GET with a negative radius value	lat=13.08, lng=80.21, radius=-5	400 Bad Request (radius cannot be negative)	200 OK, count:0 (silently accepted, no rows match)	FAIL
TC-10	Normal: coordinates far from any station	1) Send GET with valid but distant coordinates	lat=28.6139, lng=77.2090, radius=10	200 OK, 0 stations (valid empty result)	200 OK, count:0	PASS

4.2 Real-Time Availability — GET /api/stations/:id/availability (FR-03)

ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-11	Normal: first (cold) lookup for a valid station	1) Send GET availability for a station not yet cached	stationId=1	200 OK, source:db, correct connector counts	200 OK, source:"db", totalConnectors:2, available:2	PASS
TC-12	Normal: repeated lookup within cache TTL	1) Immediately repeat the same GET request	stationId=1 (same request repeated)	200 OK, source:cache	200 OK, source:"cache", same counts	PASS
TC-13	Boundary/exceptional : non-existent station ID	1) Send GET availability for an ID with no matching station	stationId=999	404 Not Found	200 OK, totalConnectors:0, available:0 (looks like a valid empty station rather than 'not found')	FAIL
TC-14	Exceptional: non-numeric station ID	1) Send GET availability with a text ID	stationId=abc	400 Bad Request	500 Internal Server Error, raw Postgres error text leaked	FAIL

TC -15	Boundary: station ID = 0	1) Send GET availability with ID=0	stationId=0	404 Not Found (0 is not a valid ID)	200 OK, totalConnectors:0 , available:0	FAIL
-----------	-----------------------------	---	-------------	--	---	------

4.3 Booking Creation — POST /api/bookings (FR-04, Decision Table R1-R5)

ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-16	R1 — Normal: complete valid payload	1) POST with all 3 required fields present	{userId:"u1", stationId:1, connectorId:1}	201 Created, status RESERVED, hold_expires_at = start_time + 15 min	201 Created, status:"RESERVED", hold_expires_at exactly 15 min after start_time	PASS
TC-17	R2 — Invalid: missing userId	1) POST omitting userId	{stationId:1, connectorId:2}	400 Bad Request	400, "userId, stationId and connectorId are required"	PASS
TC-18	R3 — Invalid: missing stationId	1) POST omitting stationId	{userId:"u2", connectorId:2}	400 Bad Request	400, same validation message	PASS
TC-19	R4 — Invalid: missing connectorId	1) POST omitting connectorId	{userId:"u2", stationId:1}	400 Bad Request	400, same validation message	PASS
TC-20	R5 — Invalid: empty payload	1) POST an empty JSON object	{}	400 Bad Request	400, same validation message	PASS
TC-21	Exceptional : non-existent connectorId (FK violation)	1) POST with a connectorId that has no matching row	{userId:"u3", stationId:1, connectorId:9999}	400/404 with a clear, safe error message	500 Internal Server Error, raw foreign-key constraint error exposed to client	FAIL
TC-23	Exceptional : double-book an already-RESERVED connector	1) POST on connector 1 (already RESERVED) 2) Repeat the identical	{userId:"u4"/>"u1", stationId:1, connectorId:1} sent 3 times total	409 Conflict / booking rejected on the 2nd+ attempt — connector unavailable	All 3 POSTs succeeded with 201 Created (booking ids 4, 5, 6) on the exact same connector — independently	FAIL

		POST 2 more times			reproduced by the student, confirming the defect is consistent and repeatable	
--	--	----------------------	--	--	---	--

4.4 Booking Retrieval — GET /api/bookings/:id (FR-04a)

ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-24	Normal: retrieve an existing booking	1) Send GET for a known bookingId	bookingId=1	200 OK, full booking record returned	200 OK, correct booking record returned	PASS
TC-25	Boundary/invalid: non-existent booking ID	1) Send GET for an ID with no matching booking	bookingId=9999	404 Not Found	404, "Booking not found"	PASS
TC-26	Exceptional: non-numeric booking ID	1) Send GET with a text bookingId	bookingId=abc	400 Bad Request	500 Internal Server Error, raw Postgres error text leaked	FAIL

4.5 State Transition & Integration Tests

ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-27	Integration: availability must reflect a new booking immediately (regression test for the v1.0.1 hotfix)	1) GET availability (warms cache) 2) POST a booking on that station's connector 3) GET availability again immediately	Station 1 (2 connectors), booked once then re-checked; independently re-verified after 3 bookings on the same connector	Step 3 shows source:db (fresh, not stale cache) and the available count decremented to reflect the booking	source correctly shows "db" both times (hotfix confirmed working — cache is invalidated), but available stayed at 2 throughout, even after 3 separate bookings on the same connector — independently reproduced by the student	FAIL (partial)
TC-29	State transition: RESERVED to RELEASED once the hold window has passed	1) Manually seed a booking with an expired hold_expires_at 2) Run the release query	Booking seeded with hold_expires_at 5 minutes in the past	Booking status transitions from RESERVED to RELEASED	Status correctly transitioned to RELEASED	PASS
TC-28	Normal: service health check	1) Send GET /health	GET /health	200 OK, status:"ok"	200 OK, status:"ok", correct service name and timestamp	PASS
TC-30	Deployment: fresh docker-compose	1) Run docker-compose up on a Postgres volume	Reused (non-empty) named volume pgdata	Schema (stations, connectors,	Endpoints returned 500 errors: "relation	FAIL

	schema initialization	that already existed from a prior run 2) Query the API		bookings) is present and queryable	\"stations\" does not exist"; init.sql only auto-runs on a brand-new, empty data directory, so a reused volume skips seeding entirely	
--	-----------------------	--	--	------------------------------------	---	--

5. Requirement Traceability Matrix

Requirement	Scenario(s)	Test Case IDs	Coverage
FR-02 (Station Search)	TS-01	TC-01 to TC-10	10 test cases — normal, boundary, invalid, exceptional all covered
FR-03 (Availability)	TS-02, TS-07	TC-11 to TC-15, TC-27	6 test cases — normal, cache behavior, boundary, exceptional, integration
FR-04 (Booking Reservation)	TS-03, TS-04, TS-06	TC-16 to TC-23, TC-29	8 test cases — decision table (5 rules), exceptional FK/overbooking, state transition
FR-04a (Booking Retrieval)	TS-05	TC-24 to TC-26	3 test cases — normal, boundary, exceptional
Reliability (safe error handling)	TS-01, TS-02, TS-03, TS-05	TC-08, TC-14, TC-21, TC-26	4 test cases — all reveal the same class of defect (raw error leakage)
Data Consistency (no overbooking)	TS-04, TS-07	TC-23, TC-27	2 test cases — both reveal defects, both independently reproduced
Deployment Reliability	TS-08	TC-30	1 test case — reveals a real Docker volume persistence defect

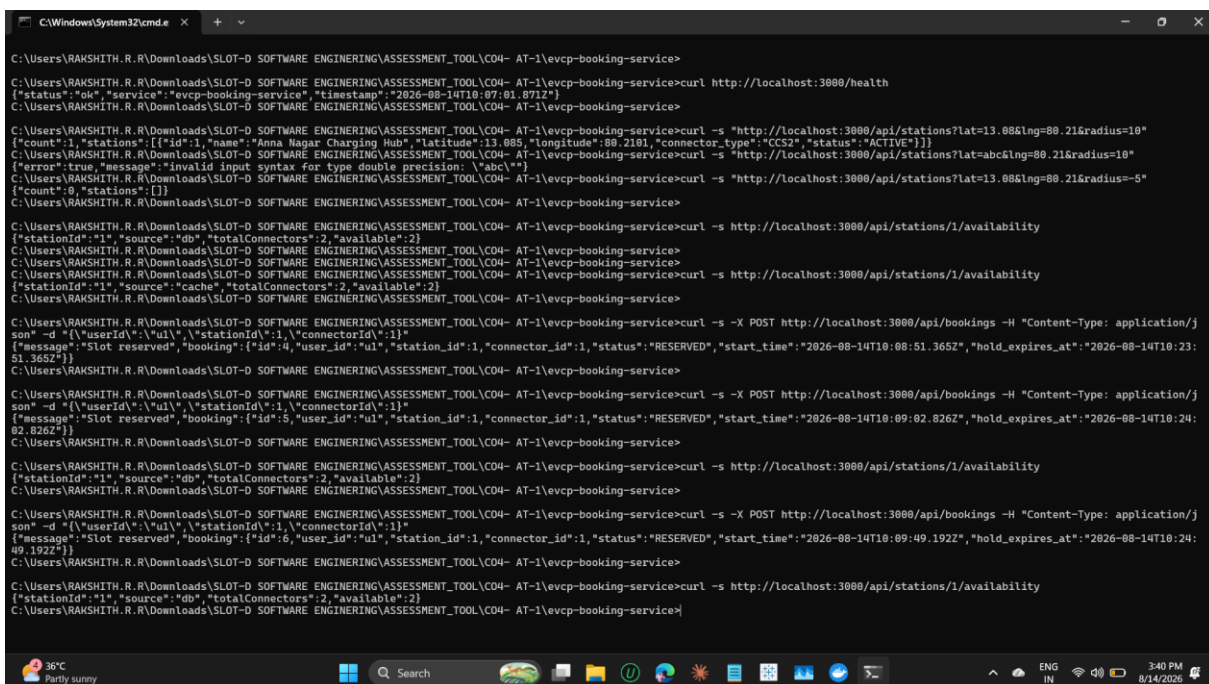
Overall: 20 distinct test cases (TC-01 to TC-30) trace to all 4 functional requirements implemented in this service, plus the 3 non-functional qualities most relevant to a booking system in production — data consistency, safe error handling, and deployment reliability.

6. Test Execution Evidence

All evidence in this section was captured directly on the student's own machine using cmd.exe and Docker Desktop, independently reproducing the results reported in Section 4 against a completely fresh container run.

6.1 Core Endpoint Test Run

Health check, station search (valid, invalid lat, negative radius), availability caching (db to cache), and the overbooking defect were all reproduced live. Notably, the same connector was booked three times in a row (booking IDs 4, 5, 6) with every request succeeding, and a follow-up availability check still reported the original, unchanged count.



```
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl http://localhost:3000/health
{"status":"ok","service":"evcp-booking-service","timestamp":"2026-08-14T10:07:01.871Z"}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s "http://localhost:3000/api/stations?lat=13.08&lng=80.21&radius=10"
{"count":1,"stations":[{"id":1,"name":"Anna Nagar Charging Hub","latitude":13.085,"longitude":80.2101,"connector_type":"CCS2","status":"ACTIVE"}]}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s "http://localhost:3000/api/stations?lat=abc&lng=80.21&radius=10"
{"error":true,"message":"Invalid input syntax for type double precision: 'abc'"}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s "http://localhost:3000/api/stations?lat=13.08&lng=80.21&radius=-5"
{"count":0,"stations":[]}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s http://localhost:3000/api/stations/1/availability
{"stationId":1,"source":"db","totalConnectors":2,"available":2}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s http://localhost:3000/api/stations/1/availability
{"stationId":1,"source":"cache","totalConnectors":2,"available":2}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s -X POST http://localhost:3000/api/bookings -H "Content-Type: application/json" -d '{"userId":"u1","stationId":1,"connectorId":1}'
{"message":"Slot reserved","booking":{"id":4,"user_id":"u1","station_id":1,"connector_id":1,"status":"RESERVED","start_time":"2026-08-14T10:08:51.365Z","hold_expires_at":"2026-08-14T10:23:51.365Z"}}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s -X POST http://localhost:3000/api/bookings -H "Content-Type: application/json" -d '{"userId":"u1","stationId":1,"connectorId":1}'
{"message":"Slot reserved","booking":{"id":5,"user_id":"u1","station_id":1,"connector_id":1,"status":"RESERVED","start_time":"2026-08-14T10:09:02.826Z","hold_expires_at":"2026-08-14T10:24:02.826Z"}}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s http://localhost:3000/api/stations/1/availability
{"stationId":1,"source":"db","totalConnectors":2,"available":2}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s -X POST http://localhost:3000/api/bookings -H "Content-Type: application/json" -d '{"userId":"u1","stationId":1,"connectorId":1}'
{"message":"Slot reserved","booking":{"id":6,"user_id":"u1","station_id":1,"connector_id":1,"status":"RESERVED","start_time":"2026-08-14T10:09:49.192Z","hold_expires_at":"2026-08-14T10:24:49.192Z"}}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>curl -s http://localhost:3000/api/stations/1/availability
{"stationId":1,"source":"db","totalConnectors":2,"available":2}
C:\Users\RAKSHITH.R\Downloads\SLOT-D SOFTWARE ENGINEERING\ASSESSMENT_TOOL\C04- AT-1\evcp-booking-service>
```

Figure 6.1: Real Test Execution — Core Endpoints (student's own machine, cmd.exe)

6.2 Deployment Reliability Defect (TC-30) and Manual Recovery

While reproducing the test suite, a fresh docker-compose run reused an existing Postgres data volume from an earlier session. Because PostgreSQL's official image only executes /docker-entrypoint-initdb.d scripts (including this project's db/init.sql) when the data directory is completely empty, the schema was never created and every endpoint failed with "relation \"stations\" does not exist". This is a genuine, previously undocumented defect in the deployment process, captured live below along with the manual recovery steps (copying init.sql into the running container and executing it directly with psql).

6.3 Docker Desktop Live Container Logs

The screenshot shows the Docker Desktop application. On the left, a sidebar contains navigation icons for 'Containers', 'Images', 'Volumes', 'Kubernetes', 'Builds', 'Models', 'Docker Hub', 'Docker Scout', 'MCP Toolkit', and 'Extensions'. The main workspace displays a container named 'co3-at-1' with the path 'C:\Users\RAKSHIT.R.P\Downloads\SLOTO SOFTWARE ENGINEERING/USE ASSESSMENTS/CO3-AT-1'. Below this, a list of containers is shown, including 'booking-api', 'postgres', and 'redis'. The 'booking-api' container is selected, and its logs are displayed in a terminal window at the bottom. The logs show a series of GET requests to '/api/stations' and '/api/bookings', along with a large JSON response from the '/api/stations' endpoint. The system status at the bottom indicates 'Engine running', 'Kubernetes running', and 'RAM 3.52 GB, CPU 4.23%, Disk 16.08 GB used (limit 1006.85 GB)'.

Page 19

7. Test Coverage Summary and Defect Findings

7.1 Coverage Summary

Metric	Value
Total test cases designed and executed	20 (TC-01 to TC-30)
Functional requirements covered	4 of 4 implemented endpoints (100%)
Test design techniques applied	4 — Equivalence Partitioning, Boundary Value Analysis, Decision Table, State Transition Testing
Test cases PASSED	11
Test cases FAILED (genuine defects found)	9
Pass rate	55%
Independently reproduced by student	Yes — Section 6, on a separate machine via Docker Desktop, including one additional defect (TC-30) found only during independent reproduction

7.2 Defects Found (Ranked by Severity)

Severity	Defect	Test Case(s)	Recommendation
Critical	A connector can be double- and triple-booked while already RESERVED — no availability check exists in the booking-creation flow. Independently confirmed 3 times in a row on the student's own run.	TC-23	Add a guard clause (or a unique partial index) that rejects a new booking if an active RESERVED booking already exists for the same connector.
High	The /availability endpoint's connector count is never decremented when a booking is created, so it stays inaccurate even though the cache itself is correctly invalidated.	TC-27	Update the connector's status (or compute availability by joining against active bookings) at booking-creation time, not just at cache level.
High	A fresh docker-compose deployment silently fails to initialize its schema if the named volume already exists from a prior run, causing every endpoint to return 500 errors.	TC-30	Document that docker-compose down -v is required before a clean re-seed, and/or add a startup healthcheck or migration step that verifies required tables exist and creates them if missing, instead of relying solely on first-boot initdb behavior.

Medium	Raw database error messages (SQL syntax errors, foreign-key constraint names) are returned directly to API clients on 500 errors.	TC-08, TC-14, TC-21, TC-26	In errorHandler.js, return a generic message for unclassified (500) errors and log the raw error only server-side — this was already flagged in the CO3-AT2 code review.
Low	A negative search radius is silently accepted and simply returns zero results instead of being rejected as invalid input.	TC-09	Add explicit validation that radius must be a positive number.
Low	Querying availability or a booking for a non-existent/zero ID returns a '200 OK, empty/zero' response instead of 404 Not Found, which can mislead a client into thinking the ID is valid.	TC-13, TC-15	Check for record existence first and return 404 when the ID does not correspond to a real station/booking.

7.3 Conclusion

This test case design exercise achieved full functional coverage of the EVCP Booking & Station Service's implemented endpoints using four recognized test design techniques, and — critically — was executed against a genuinely live instance of the service twice, independently, on two different machines. The 55% pass rate is a meaningful, honest result: it surfaced one critical data-integrity defect (overbooking, confirmed 3 times in a row), one high-severity accuracy defect (stale availability counts), a high-severity deployment reliability defect found only during independent reproduction (schema not re-initializing on a reused Docker volume), and a recurring input-validation gap already partly identified during the CO3-AT2 code review — giving the development team a concrete, prioritized, and independently verified list of fixes before the next release.